



PUBLIC

ABAP Cloud API Enablement Guidelines for SAP S/4HANA Cloud Private Edition, and SAP S/4HANA

Guidelines for administrators and ABAP developers

Version: 3.0

Date: August 2025

Owner: SAP BTP ABAP Product Management



ACTIVITY LOG

Version 3.0

Chapter	Note
All	Transformation from 3-tier extensibility to clean core extensibility based on clean core level concept
2.2	Changed link to Customer Influence channel
2.1.1	Minor changes regarding software components for tier 2
2.3.1.1	Added paragraph regarding custom DDIC search helps
2.3.4.4	Chapter added
2.4	Added reference to classic APIs
All	Adjusted to updated guidance in ABAP Extensibility guide

Table of Contents

ACTIVITY LOG	2
VERSION 3.0	2
INTRODUCTION	4
DEVELOPMENT MODELS IN SAP S/4HANA CLOUD PRIVATE EDITION AND SAP S/4HANA	4
WHAT'S IN?	5
EXPECTATIONS AND OBLIGATIONS	5
DEVELOPMENT GUIDELINES FOR ABAP CLOUD API ENABLEMENT	6
SETUP RECOMMENDATIONS	6
<i>Software Component</i>	6
<i>Packages</i>	6
<i>ABAP Test Cockpit</i>	6
<i>Authorizations</i>	7
<i>Development environment</i>	7
BASIC PRINCIPLES	7
BUILDING WRAPPERS	8
<i>General Aspects</i>	8
<i>Mitigate Missing Business Logic and Data Access APIs</i>	9
<i>Mitigate Missing SAP HANA Features</i>	10
<i>Further Remarks</i>	11
OBJECTS SUITED TO BE WRAPPED	12
CONSUMPTION IN ABAP CLOUD DEVELOPMENT	13
USAGE	13
ERROR LOGS	13
REPLACEMENT AND REFACTORING	14

INTRODUCTION

Development models in SAP S/4HANA Cloud Private Edition and SAP S/4HANA

The extensibility guidelines for SAP S/4HANA Cloud Private Edition and SAP S/4HANA on-premise provided in the [ABAP Extensibility Guide](#) propose co-existence of ABAP Cloud development and classic ABAP development models whereby ABAP Cloud development should be default choice for all new extensions and classic ABAP development should be used only for legacy code which cannot be transformed to ABAP Cloud or for new extensions in application areas where ABAP Cloud is not applicable.

The overview and interplay of ABAP Cloud development and classic ABAP development is provided on the following picture.

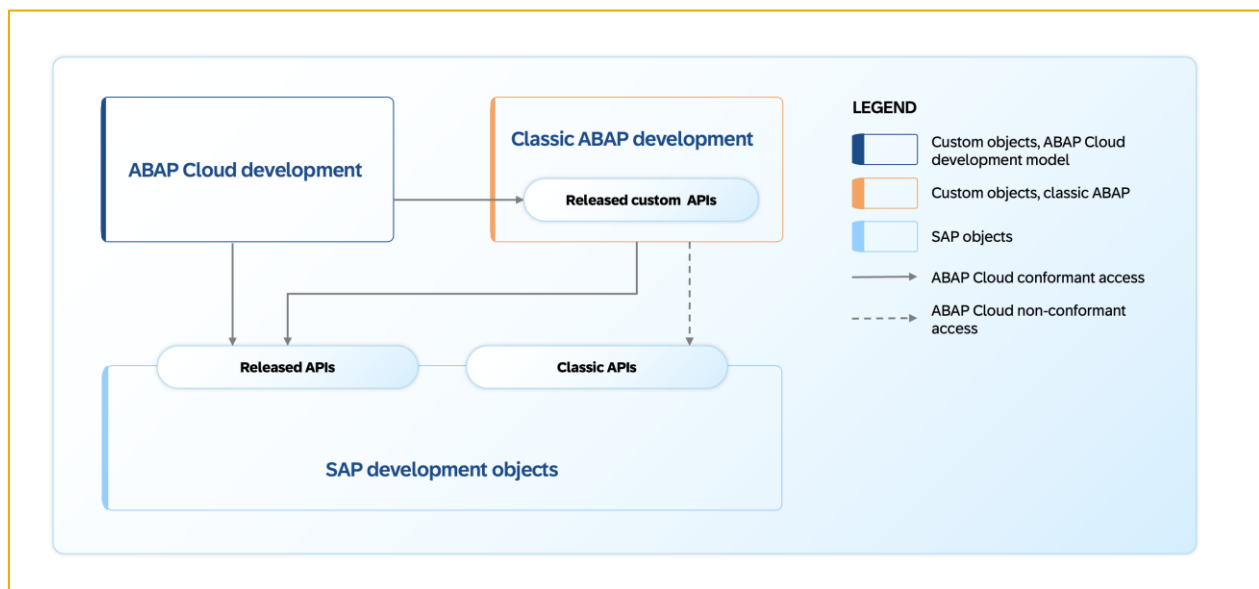


Figure 0.1 - Interplay between ABAP Cloud development and classic ABAP development

The custom code which follows ABAP Cloud development rules can by default only access the publicly released SAP APIs. They are given by the SAP development objects that are released for cloud development. In case APIs for business logic or data access are missing in the publicly released SAP APIs, wrappers for non-released SAP objects can be built in classic ABAP based on classic APIs and [released for ABAP Cloud¹](#). SAP provides explicit recommendations for classic APIs that are suited to be enabled for ABAP Cloud. The concept of classic APIs is explained in detail in the [ABAP Extensibility Guide](#). In this way ABAP Cloud API enablement constitutes custom interfaces to SAP development objects (released custom APIs). Access to the released SAP development objects and to the custom wrappers is ABAP Cloud conform (solid arrows), while access to classic APIs is not ABAP Cloud conform (dashed arrow). If applicable, for classic ABAP developments it is also recommended to use the publicly released SAP APIs or custom SAP interfaces for new implementations. While ABAP Cloud development rules are enforced via syntax and runtime checks, the classic ABAP developments shall be controlled by the ABAP test cockpit.

¹ Short notation for releasing the object under the contract C1 with visibility "Use in Cloud Development".

In this guide, we provide more information regarding the setup of wrapper developments for ABAP Cloud API enablement, about building wrappers for non-released SAP objects and about consuming these wrappers in ABAP Cloud development.

What's In?

The purpose of ABAP Cloud API enablement is to mitigate missing publicly released SAP APIs while sticking to the ABAP Cloud development rules and guidelines as much as possible.

With ABAP Cloud API enablement, you can

- mitigate missing released SAP APIs that are required for ABAP Cloud development.
- control the dependencies to non-released SAP development objects by building a clear custom interface between SAP and custom code (released custom APIs).

Expectations and Obligations

Although objects that are suited for building wrappers to mitigate missing released SAP APIs are almost stable, there is no such guarantee for upgrade-stability as we have for the released SAP APIs. Hence, anything developed for ABAP Cloud API enablement needs to be reviewed and tested during upgrade projects.

To offer a broad scope of mitigation possibilities during ABAP Cloud API enablement, possible violations of strict ABAP Cloud rules in legacy objects need to be tolerated. As a consequence, it is possible that ABAP Cloud development using wrappers does not fulfill all qualities as a pure ABAP Cloud development (e.g. regarding transactional consistency, see the blog posts [The SAP LUW in ABAP Cloud](#) and [Using BAPIs in RAP](#)). It is the obligation of the customer to ensure correct functionality in these scenarios. SAP offers tools to identify possible issues.

The guidelines for ABAP Cloud API enablement given in this paper are not enforced by SAP, but they are a recommendation that may be adapted to meet future needs. It is the customer's responsibility to establish these or similar guidelines in his company.

DEVELOPMENT GUIDELINES FOR ABAP CLOUD API ENABLEMENT

In this chapter some recommendations regarding the technical setup of wrapper development for ABAP Cloud API enablement are provided, followed by guidelines regarding the development of wrappers. Finally, proposals for SAP APIs to be wrapped are made.

Setup Recommendations

The basic setup of development with ABAP Cloud and classic ABAP is explained in the [ABAP Extensibility Guide](#). Here, we summarize the setup of development of wrappers for ABAP Cloud API enablement.

Software Component

In ABAP Cloud API enablement access to non-released SAP objects is required. Hence, the created objects must be in a software component with default language version Standard ABAP² and cannot be in the same software component as ABAP Cloud development. It is not required to create a software component solely for wrapper objects for ABAP Cloud API enablement. It is up to own policy and choice whether to use packages in HOME or an existing software component with default language version Standard ABAP, or whether to [create a dedicated software component](#) for wrapper objects with or without namespace.³

Packages

It is recommended to create dedicated packages or a package tree containing all objects required for building wrappers. The transport layer for wrapper packages must correspond to the transport layer of the ABAP Cloud developments to ensure that the wrappers are transported along the same transport routes as the objects using them.

ABAP Test Cockpit

The wrapper development for ABAP Cloud API enablement shall be as close to ABAP Cloud as possible, meaning that all objects and coding should be created as it would be required in ABAP Cloud, except for single dedicated violations. Hence it is recommended to check for any violation of strict ABAP Cloud principles using ABAP test cockpit (ATC). The information on how set up the ABAP test cockpit check variant to optimally serve the governance approach for all developments done in classic ABAP, including wrapper development, is provided in the [ABAP Extensibility Guide](#).

If released SAP APIs are provided and replace non-released objects that are used in wrappers, the ABAP test cockpit will detect this and indicate refactoring options.

Further information on API snapshots required for the stability of wrappers is give in the [SAP Help Portal documentation](#).

² *Standard ABAP* is the technical name of the language version in the classic ABAP development model.

³ [SAP Note 3319469](#) is required in this case.

Authorizations

It is recommended to create a dedicated new developer role for developers that are authorized and skilled to develop mitigations for missing SAP APIs as wrappers. A good starting point is to use the role template for ABAP Cloud (SAP_BC_ABAP_DEVELOPER_5) and add the authorization for classic ABAP. This is done using the authorization object S_ABPLNGVS. More information can be found in the SAP Help Portal documentation [Configure Authorizations](#). The role template includes the authorizations for all object types that are part of ABAP Cloud development. Further object types are not required for wrapper development⁴.

Development environment

ABAP development tools for Eclipse (ADT) must be used for wrapper development. Several modern development object types like CDS view entities, behavior definitions and others can only be edited in ADT.

Basic Principles

The general idea for developing wrappers is:

- Create wrapper objects in classic ABAP
- Implement the objects and make use of non-released SAP development objects based on SAP classic APIs.
- [Release](#) the required objects for ABAP Cloud. These objects form your custom SAP interface (released custom API).

In addition to the governance possibilities explained above, the wrappers encapsulate the usage of SAP development objects, which can ease later refactoring and allow developers to concentrate test efforts to dependencies that are potentially not upgrade-stable during SAP software upgrades.

In general, the wrapper development shall be as compatible to ABAP Cloud as possible. That means that only object types available in ABAP Cloud shall be developed and the implementation shall follow ABAP Cloud syntax rules. If set up as recommended before, any violation of that will result in ABAP test cockpit errors for which dedicated exemptions need to be requested.

Finally, the need to develop a wrapper can indicate that a released SAP API is missing. SAP encourages customers and partners to make use of the [Customer Influence channel](#) for released SAP APIs to request the missing API.

⁴ Authorizations for specific object types are steered by authorization object S_DEVELOP.

Building Wrappers

General Aspects

ABAP Dictionary Artefacts

SAP does not release all ABAP Dictionary artefacts (domains, data elements, structures, table types, ...). However, where types or structures are of central importance, or if they are used to type the key fields of released CDS views, these types should be released and reused in custom development. If such a dictionary artefact is missing, it should be requested via the [Customer Influence channel](#). Still, the question arises of how to deal with non-released dictionary artefacts when building wrappers. The recommendations are:

- If the types are only required in the business logic for calling the SAP API and in ABAP Cloud development for calling the wrapper, then the types should be defined locally as part of the wrapper. This can be done using the TYPES statement in wrapper classes or interfaces (an example is provided in the [ABAP Extensibility Guide](#)). This way the wrapper class or interface is self-contained, no further dictionary artefacts need to be created or released for the usage of the wrapper in ABAP Cloud development.
- If the types are required in ABAP Cloud development to define further ABAP dictionary artefacts (data elements, tables, ...) they should be recreated as custom dictionary artefacts in ABAP Cloud directly⁵.

ABAP dictionary search helps are not part of ABAP Cloud and will not be released by SAP. Instead, search helps are provided by corresponding CDS view entities with the required annotations. If such a released value help view entity is missing, the recommendation is to create a custom value help view on top of the corresponding table as described in the next section. If existing custom code-based search helps shall be reused in ABAP Cloud development, the corresponding function module needs to be wrapped for ABAP Cloud API enablement as explained below. Then a CDS based search help can be created in ABAP Cloud by a custom entity which used the wrapped function module in the implementation.

Authorizations

If PFCG roles are created, the role administrator needs to know which authorizations are required for the desired activities. While all objects released for ABAP Cloud have a clear authorization signature and the corresponding authorization objects are released, this does not necessarily hold true anymore if a wrapper is used. In wrapped non-released SAP APIs, there can be authorization checks on objects that are not released. If known, the recommendation is to provide corresponding authorization default values (transaction SU22) for wrappers when [releasing the wrapper object for ABAP Cloud](#). In addition, the developer should create for each ABAP Cloud service (that uses non-released authorization checks at runtime) a [Variant for Default Data in SU22](#) with the non-released authorization objects from wrappers of ABAP Cloud API enablement.

Later, role administrators can construct PFCG roles for ABAP Cloud extensions based on the authorization default values of released SAP APIs used in ABAP Cloud and their corresponding default variants in custom wrapper

⁵ The dictionary artefacts created in ABAP Cloud development can be used in wrapper development if they are released for cloud development.

development. Authorization objects themselves (your own authorization objects in classic ABAP or non-released SAP authorization objects) are not wrapped and do not need to be released for ABAP Cloud. Moreover, PFCG roles are customizing and are not part of the ABAP development.

Connectivity

To consume HTTP services, SOAP services or RFC enabled function modules in ABAP Cloud developments, it is required to access the corresponding SM59 destination or logical port. Currently, there are no publicly released SAP APIs available for that. Instead, we recommend writing different wrapper classes around CL_OUTBOUND_PROVIDER_HTTP (or _RFC, or _SOAP) for each use case. More information is provided in the SAP Help Portal documentation [Developing External Service Consumption \(Outbound Communication\)](#). Starting with the SAP S/4HANA 2025 release it will become possible to establish HTTP and RFC communication in ABAP Cloud using so-called communication targets, for SOAP it will become possible to establish communication without a wrapper.

Transactional Consistency

For wrappers it is possible to call SAP APIs and use ABAP language statements that are not allowed in strict ABAP Cloud rules in ABAP Cloud development. In pure ABAP Cloud development these violations would result in syntax or runtime errors. If these rules are violated in wrappers however, they are tolerated because it cannot be guaranteed that all legacy APIs used in the wrappers follow the ABAP Cloud rules⁶. Consequently, it is the responsibility of the wrapper developer together with the ABAP Cloud developers to ensure correct functionality of the wrappers in ABAP Cloud developments. In the last chapter of this paper, it is shown how to detect possible pitfalls.

Mitigate Missing Business Logic and Data Access APIs

The following non-released objects may be wrapped to mitigate missing publicly released SAP APIs:

TYPE OF SAP OBJECT	STEPS TO CREATE A WRAPPER
RAP⁷ BUSINESS OBJECT	• Create a self-contained wrapper class (details below). • Provide methods for each access to the RAP BO and call them programmatically via Entity Manipulation Language (EML). • Release the developed class for ABAP Cloud and define SU22 values.
FUNCTION MODULE CLASS / INTERFACE	• The SAP GUI transaction ACO_PROXY can be used for automated wrapper generation • Create a self-contained wrapper class (details below). • Provide methods for each access of a non-released SAP function module or class/interface and call it in the implementation.

⁶ The ABAP statements COMMIT WORK and ROLLBACK WORK are never tolerated in a [strict transactional context](#) and will always lead to a runtime error, also if they are executed in a wrapper.

⁷ Short for ABAP RESTful application programming model

	• Release the developed class for ABAP Cloud and define SU22 values.
CDS VIEW / TABLE	• Create a CDS View Entity on top of the non-released SAP table or view. Note: • Prefer CDS view entities over tables. Dictionary views are not supported. • Prefer use-case specific CDS views instead of complex ones. • Decouple the CDS view entity wrapper from underlying CDS annotations by <code>@Metadata.ignorePropagatedAnnotations: true</code>. Re-insert only the annotations required in the wrapper. • <u>Create a DCL</u> for the newly created CDS view entity inheriting from the SAP DCL or defining own authorization checks if required. • Release the developed CDS view entities for ABAP Cloud and define SU22 values.

Table 0.1 - Non-released objects that can be wrapped to mitigate missing publicly released SAP APIs.

For the development of wrapper classes, we recommend:

- Declare relevant types locally to decouple from non-released dictionary artefacts (see recommendation regarding ABAP dictionary artefacts above).
- Use class-based exceptions inheriting from CX_STATIC_CHECK, use factory methods instead of public constructors, declare wrapper classes as FINAL.
- Follow Clean ABAP Guidelines.

It is good practice to declare the public types and methods in an interface. Note that when releasing the wrapper class for ABAP Cloud development, consistency of the wrapper is enforced via ABAP test cockpit. Hence, along with the wrapper class the corresponding exception class as well as referenced types or interfaces must be released. Information on how to release development objects for ABAP Cloud can be found in the SAP Help Portal documentation Setting the API Release State. Once a wrapper is released, it must only be changed compatibly in order not to harm consuming applications. To that end the API snapshots and corresponding ABAP test cockpit checks must be used (as explained in above section), to enforce the contract rules.

Mitigate Missing SAP HANA Features

Besides usual ABAP CDS view entities it is possible to develop CDS table functions or ABAP Managed Database Procedures (AMDP) to work natively with SAP HANA and benefit from code-pushdown. However, in ABAP Cloud development only a limited scope is released (READ-ONLY). In custom wrapper development, the features that are not yet offered in ABAP Cloud can be mitigated. This mitigation is not achieved by a wrapper of an SAP API, but by developing the required object for ABAP Cloud API enablement and releasing it for ABAP Cloud.

USAGE IN ABAP CLOUD AS:	STEPS
CDS TABLE FUNCTION	• Create the CDS table function in classic ABAP. • Create the implementing ABAP class in classic ABAP and implement the required interface using all desired SAP HANA features. • Create a CDS view entity using the CDS table function. • Release the CDS view entity for ABAP Cloud.
AMDP METHOD	• Create the class implementing the AMDP method in classic ABAP. • Implement the methods using all desired SAP HANA features. • Release the class for ABAP Cloud.

Table 0.2 - Usage of code pushdown capabilities in ABAP Cloud development

The released objects can be used without any restrictions in ABAP Cloud development.

Further Remarks

Analytics

In ABAP Cloud development we have analytical models purely based on CDS. Classic business warehouse (BW) models are not part of ABAP Cloud. It is not possible to wrap classic BW models to enable them for CDS based analytics in ABAP Cloud development. That is because BW models are generated in each system, whereas CDS-based analytical models are developed in the development system and are transported through the landscape. This disjoint lifecycle of the two concepts makes it impossible to manage dependencies safely. That's why classic BW models are always located in classic ABAP development. On the other hand, CDS-based analytical models are located in ABAP Cloud development. If required CDS based analytical models are not released, they can be wrapped as described above.

Objects released for key user extensibility but not for developer extensibility in ABAP Cloud

Objects released for key user extensibility are also upgrade-stable. They follow the same stability contract as for developer extensibility in ABAP Cloud. There are objects that are released for key user extensibility only, but not for developer extensibility in ABAP Cloud. This can occur for example, in the following domains:

- Analytical queries⁸:
Released analytical queries can be used in analytical key user extensibility scenarios only. There is no use case of

⁸ Identified by annotation `@ObjectModel.ModelingPattern = ANALYTICAL_QUERY` or `@Analytics.query: true`.

re-using analytical queries in ABAP cloud development. Customers can create their own analytical queries based on the underlying released CDS views (base or cube views).

- CDS views for data replication scenarios⁹:

These views are for use in data replication tools and must not be used in ABAP Cloud development.

- CDS views for forms and email output¹⁰:

These views are for use in output management key user tools and must not be used in ABAP Cloud development.

Note that the list is not complete, it contains the most important domains and cases only. If an object not falling under these categories is missing a release for ABAP Cloud development, a corresponding requirement shall be opened in the [Customer Influence channel](#).

Non-released BADIs

Non-released BADIs shall not be implemented in wrappers for ABAP Cloud API enablement but are part of classic ABAP development. If for the implementation of a released BADI in ABAP Cloud an API is missing, a wrapper around that API can be used.

Access to configurations maintained via SM30

The options to develop business configurations are explained in the [ABAP Extensibility Guide](#). If SM30 based business configurations shall be used in ABAP Cloud development, a CDS view shall be created as wrapper on top of the configuration table as explained in section 2.3.2.

Objects Suited to be Wrapped

In general, only objects with API character shall be wrapped, but not just any SAP framework code or internal form routines. Suitable objects are for example:

- BAPI function modules (see transaction BAPI). Information about the usage of BAPIs in an ABAP Cloud RAP BO is provided in the blog post [Using BAPIs in RAP](#).
- Externally released function modules.
- Non-released RAP BOs (look for objects of type BDEF in ADT).
- CDS views or tables, with preference for CDS views.

SAP provides explicit recommendations for classic APIs that are suited to be enabled for ABAP Cloud. More information is provided in the [ABAP Extensibility Guide](#).

⁹ Can be identified by the respective CDS annotations, e.g., `@ObjectModel.supportedCapabilities: #EXTRACTION_DATA_SOURCE`).

¹⁰ Can be identified by the respective CDS annotations, e.g., `@ObjectModel.ModelingPattern: #OUTPUT_EMAIL_DATA_PROVIDER` or `#OUTPUT_FORM_DATA_PROVIDER`.

CONSUMPTION IN ABAP CLOUD DEVELOPMENT

In this chapter it is explained how to use wrappers in ABAP Cloud development, how to detect issues in wrappers that might have impact on the correct functionality of your ABAP Cloud development and how to refactor if a public SAP API is made available.

Usage

Wrapper objects implemented for ABAP Cloud API enablement can be used in ABAP Cloud development in the same way as release SAP APIs, since they are released for ABAP Cloud.

To determine and grant the necessary authorizations, role administrators can use comparison with SU22 data (SU25) to copy the SU22 data delivered by the developer with the release of the wrapper and the default variant into the customer-specific tables (SU24 data). See [From the Programmed Authorization Check to a Role | SAP Help Portal](#) (transaction SU25 step 2a and 2b). This way, role administrators can construct PFCG roles for ABAP Cloud extensions based on the authorization default values of public SAP APIs used in ABAP Cloud and their corresponding default variants in classic ABAP.

Additional information: For default variants it is not possible to set the check indicator: *Do not check*. This is only possible when maintaining the authorization default values. If a non-released authorization object shall not be checked for your ABAP Cloud service, then add the corresponding authorization object to the authorization default values of the service in the SU24 transaction and set the check indicator: *Do not check*.

Error Logs

There will be no syntax or runtime errors if wrappers violate strict ABAP Cloud rules as explained before. Instead, violations can be logged and used to detect possible issues and improve the wrappers and their usage.

- Any violation of transactional consistency rules [of the SAP LUW](#) will be logged via checkpoint group CC_STMT.
- This checkpoint group can be activated using transaction SAAB and the recorded logs can be inspected.

There are two recommended options:

- Activation of the checkpoint group for dedicated tests for specific users and time frames (that could e.g., be part of done criteria for ABAP Cloud developments) and analysis of all findings reported in the logs, including findings inside wrappers.
- Permanent activation of the checkpoint group in the development system, regular check upon any errors by developers.

For further analysis, the logging (e.g. cross trace) and for performance issues the profiler or ST05 transaction can be used. The overview of analysis tools is provided on the SAP Community page for [troubleshooting](#).

Replacement and Refactoring

After an SAP upgrade, the ABAP test cockpit checks in check category *Cloud Readiness* can be used to identify refactoring possibilities (see setup recommendations above). If a publicly released SAP API is available for a wrapped functionality, this SAP APIs should be used instead of unreleased objects. For refactoring in this regard, there are two options:

- Refactoring in ABAP Cloud:
 - Replace the usage of the wrapper with direct usage of the released SAP API.
 - Adapt PFCG roles to include the authorization objects of the released SAP APIs instead of the ones required for the wrapper.
 - Delete the wrapper in classic ABAP if it is not used any more.
- Refactoring in ABAP Cloud API enablement layer:
 - Replace the usage of the non-released object in the wrapper by the released SAP API.
 - Adapt the SU22 values provided when releasing the wrapper for cloud development and the default variants to only include the released authorization objects of the released SAP APIs.
 - Adapt the PFCG roles accordingly.
 - Change the ABAP language version of the wrapper object to ABAP for Cloud Development, remove any violation of strict ABAP Cloud rules.
 - Move the wrapper to the ABAP Cloud software component by assigning it to the corresponding ABAP Cloud development package¹¹.

Which option to follow, must be decided case-by-case. If the wrapper object is used multiple times in ABAP Cloud (see e.g., where-used list), option 2 can be suitable, because it minimizes the refactoring effort and keeps a central and reusable access point to SAP APIs. For single or simple usages option 1 can be appropriate as well.

Like in every refactoring project, careful continuous testing during the entire refactoring project is required. Here, this also includes the usages of the wrapper or SAP APIs in ABAP Cloud development.

¹¹ The package reassignment must be on a separate transport request. The required authorizations (S_CTS_ADMI) are not included in the authorization recommendation for ABAP Cloud API enablement as explained above.